

Отчёт по лабораторной работе №6

Компьютерный практикум по статистическому анализу данных

Решение моделей в непрерывном и дискретном времени

Выполнил: Исаев Булат Абубакарович,
НПИбд-01-22, 1132227131

Содержание

1	Цель работы	1
2	Выполнение лабораторной работы.....	1
2.1	Решение обыкновенных дифференциальных уравнений	1
2.2	Модель экспоненциального роста.....	1
2.3	Система Лоренца.....	3
2.4	Модель Лотки–Вольтерры.....	5
2.5	Самостоятельное выполнение	7
3	Вывод.....	15
4	Список литературы. Библиография	15

1 Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

2 Выполнение лабораторной работы

2.1 Решение обыкновенных дифференциальных уравнений

Вспомним, что обыкновенное дифференциальное уравнение (ОДУ) описывает изменение некоторой переменной u .

Для решения обыкновенных дифференциальных уравнений (ОДУ) в Julia можно использовать пакет `diffrentialEquations.jl`.

2.2 Модель экспоненциального роста

Рассмотрим пример использования этого пакета для решение уравнения модели экспоненциального роста, описываемую уравнением, где a — коэффициент роста.

Численное решение в Julia будет иметь следующий вид, а также график, соответствующий полученному решению (рис. 1):

```
[7]: # подключаем необходимые пакеты:
import Pkg
Pkg.add("DifferentialEquations")
using DifferentialEquations

# задаём описание модели с начальными условиями:
a = 0.98
f(u,p,t) = a*u
u0 = 1.0

# задаём интервал времени:
tspan = (0.0,1.0)

# решение:
prob = ODEProblem(f,u0,tspan)
sol = solve(prob)

# строим графики:
Pkg.add("Plots")
using Plots
gr() # Важная строка!

plot(sol, linewidth=5, title="Модель экспоненциального роста",
      xaxis="Время", yaxis="u(t)", label="u(t)")
|
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, label="Аналитическое решение")

Resolving package versions...
Project No packages added to or removed from `C:\Users\Пользователь\.julia\environments\v1.12\Project.toml`
Manifest No packages added to or removed from `C:\Users\Пользователь\.julia\environments\v1.12\Manifest.toml`
Resolving package versions...
Project No packages added to or removed from `C:\Users\Пользователь\.julia\environments\v1.12\Project.toml`
Manifest No packages added to or removed from `C:\Users\Пользователь\.julia\environments\v1.12\Manifest.toml`
```

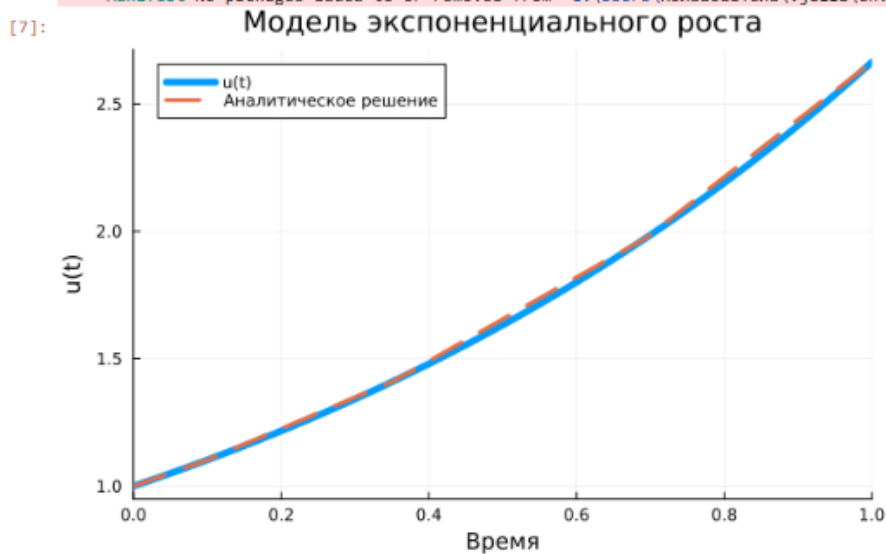


Рис. 1: График модели экспоненциального роста

При построении одного из графиков использовался вызов `sol.t`, чтобы захватить массив моментов времени. Массив решений можно получить, воспользовавшись `sol.u`.

Если требуется задать точность решения, то можно воспользоваться параметрами `abstol` (задаёт близость к нулю) и `reltol` (задаёт относительную точность). По умолчанию эти параметры имеют значение `abstol = 1e-6` и `reltol = 1e-3`.

Для модели экспоненциального роста (рис. 2):

```
[9]: # задаём точность решения:
sol = solve(prob, abstol=1e-8, reltol=1e-8)

# строим график:
plot(sol, lw=2, color="black", title="Модель экспоненциального роста", хaxis="Время", уaxis="u(t)", label="Численное решение")
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, color="red", label="Аналитическое решение")
```

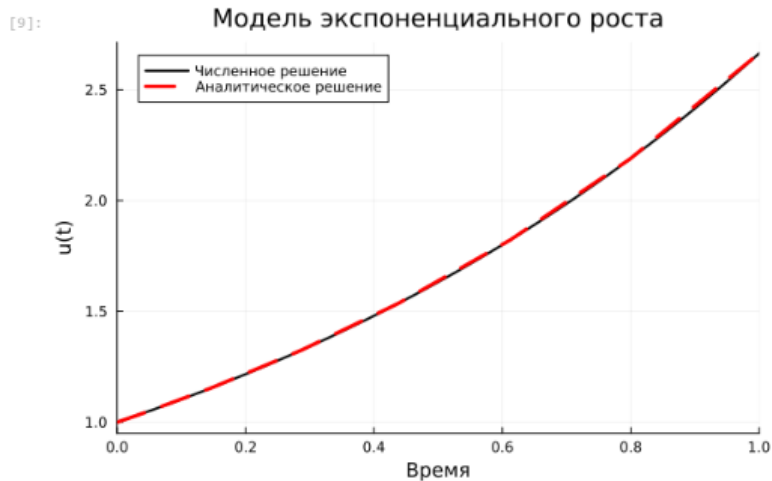


Рис. 2: График модели экспоненциального роста (задана точность решения)

2.3 Система Лоренца

Динамической системой Лоренца является нелинейная автономная система обыкновенных дифференциальных уравнений третьего порядка.

Система получена из системы уравнений Навье–Стокса и описывает движение воздушных потоков в плоском слое жидкости постоянной толщины при разложении скорости течения и температуры в двойные ряды Фурье с последующим усечением до первых-вторых гармоник.

Решение системы неустойчиво на аттракторе, что не позволяет применять классические численные методы на больших отрезках времени, требуется использовать высокоточные вычисления.

Численное решение в Julia будет иметь следующий вид (рис. 3):

```
[14]: # задаём описание модели:
function lorenz!(du,u,p,t)
    σ,ρ,β = p
    du[1] = σ*(u[2]-u[1])
    du[2] = u[1]*(ρ-u[3]) - u[2]
    du[3] = u[1]*u[2] - β*u[3]
end

# задаём начальное условие:
u0 = [1.0,0.0,0.0]
# задаём значения параметров:
p = (10,28,8/3)
# задаём интервал времени:
tspan = (0.0,100.0)
# решение:
prob = ODEProblem(lorenz!,u0,tspan,p)
sol = solve(prob)

# подключаем необходимые пакеты:
Pkg.add("Plots")
using Plots
# строим график:
plot(sol, vars=(1,2,3), lw=2, title="Аттрактор Лоренца", xaxis="x",yaxis="y", zaxis="z",legend=false)

Resolving package versions...
Project No packages added to or removed from 'C:\Users\Пользователь\.julia\environments\v1.12\Project.toml'
Manifest No packages added to or removed from 'C:\Users\Пользователь\.julia\environments\v1.12\Manifest.toml'
```

[14]: Аттрактор Лоренца

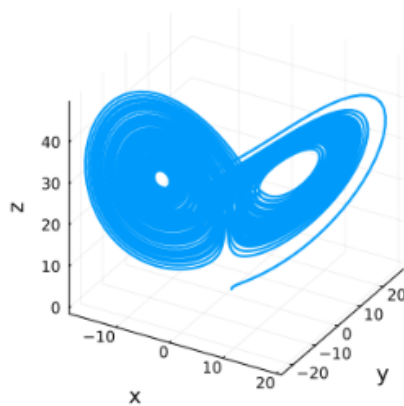


Рис. 3: Аттрактор Лоренца

Можно отключить интерполяцию (рис. 4):

```
[15]: # отключаем интерполяцию:  
plot(sol,vars=(1,2,3),denseplot=false, lw=1, title="Аттрактор Лоренца", xaxis="x",yaxis="y", zaxis="z",legend=false)
```

[15]: Аттрактор Лоренца

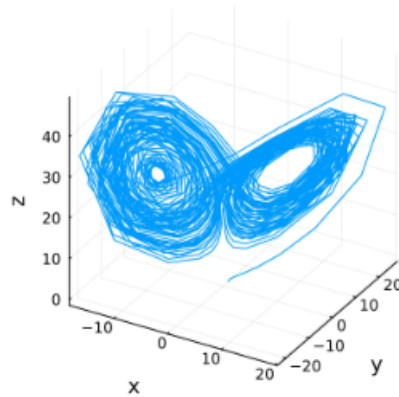


Рис. 4: Аттрактор Лоренца (интерполяция отключена)

2.4 Модель Лотки–Вольтерры

Модель Лотки–Вольтерры описывает взаимодействие двух видов типа «хищник – жертва».

Численное решение в Julia будет иметь следующий вид (рис. 5):

```
[24]: using DifferentialEquations, Plots

# задаём описание модели обычной функцией:
function lotka_volterra!(du, u, p, t)
    x, y = u
    a, b, c, d = p

    du[1] = a*x - b*x*y # dx/dt
    du[2] = -c*y + d*x*y # dy/dt
end

# задаём начальное условие:
u0 = [1.0, 1.0]

# задаём значения параметров:
p = (1.5, 1.0, 3.0, 1.0)

# задаём интервал времени:
tspan = (0.0, 10.0)

# решение:
prob = ODEProblem(lotka_volterra!, u0, tspan, p)
sol = solve(prob)

# строим график:
plot(sol, label = ["Жертвы" "Хищники"], color="black",
      linestyle=[:solid :dash], title="Модель Лотки - Вольтерры",
      xlabel="Время", ylabel="Размер популяции")
```

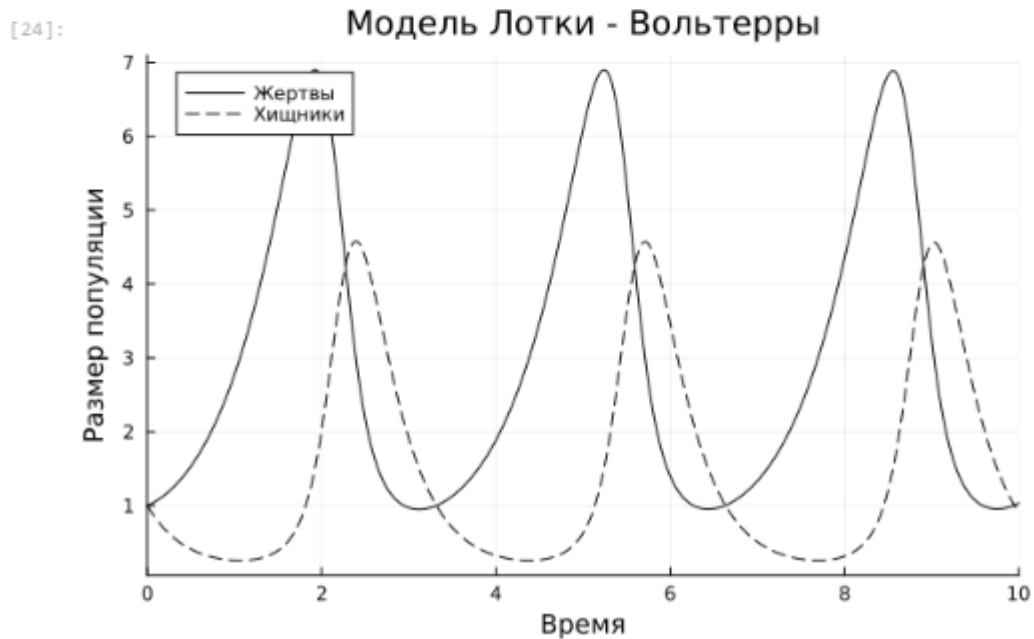


Рис. 5: Модель Лотки–Вольтерры: динамика изменения численности популяций
Фазовый портрет (рис. 6):

```
[25]: # фазовый портрет:  
plot(sol,vars=(1,2), color="black", xaxis="Жертвы",yaxis="Хищники", legend=false)
```

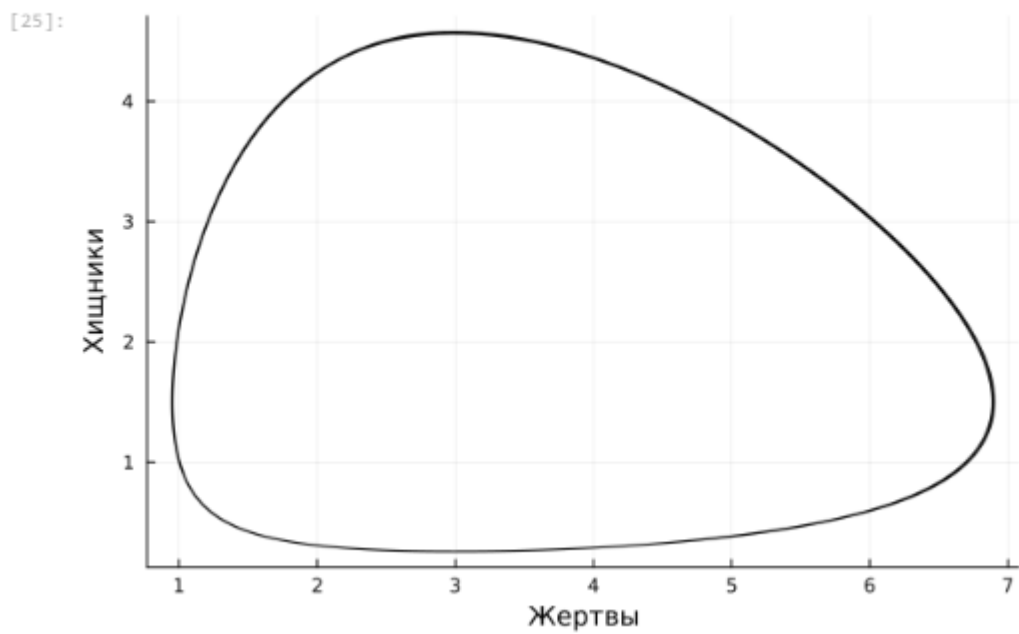


Рис. 6: Модель Лотки–Вольтерры: фазовый портрет

2.5 Самостоятельное выполнение

Выполнение задания №1 (рис. 7):

```
[26]: # ЗАДАНИЕ 1: Модель Мальтуса (экспоненциальный рост)

function malthus_model(du, u, p, t)
    a = p
    du[1] = a * u[1]
end

# Параметры: a = b - c
b = 0.3 # коэффициент рождаемости
c = 0.1 # коэффициент смертности
a = b - c # коэффициент роста = 0.2

u0_malthus = [10.0] # начальная численность популяции
tspan_malthus = (0.0, 20.0)

prob_malthus = ODEProblem(malthus_model, u0_malthus, tspan_malthus, a)
sol_malthus = solve(prob_malthus)

# График для задания 1
plot(sol_malthus, linewidth=2, title="Модель Мальтуса: Экспоненциальный рост",
     xaxis="Время", yaxis="Численность популяции", label="Численное решение",
     legend=:topleft, color=:blue)

# Аналитическое решение для сравнения
t_analytical = 0:0.1:20
x_analytical = u0_malthus[1] .* exp.(a .* t_analytical)
plot!(t_analytical, x_analytical, linewidth=2, linestyle=:dash,
     label="Аналитическое решение", color=:red)
```

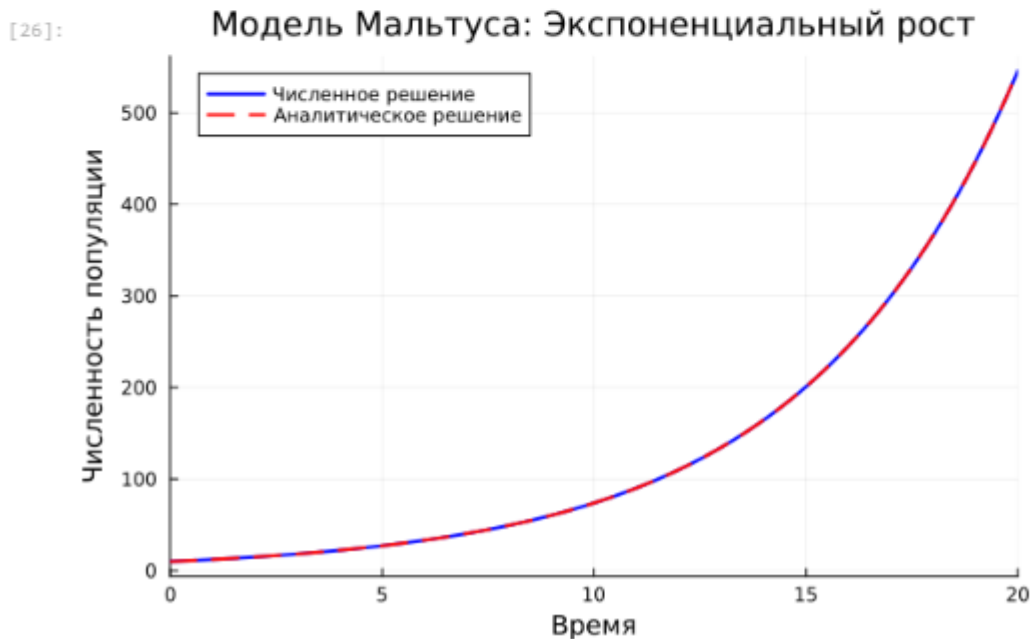


Рис. 7: Решение задания №1

Выполнение задания №2 (рис. 8):

```
[27]: # ЗАДАНИЕ 2: Логистическая модель роста

function logistic_model(du, u, p, t)
    r, k = p
    du[1] = r * u[1] * (1 - u[1]/k)
end

# Параметры
r = 0.5 # коэффициент роста
k = 100.0 # ёмкость среды
u0_logistic = [5.0] # начальная численность
tspan_logistic = (0.0, 20.0)
p_logistic = (r, k)

prob_logistic = ODEProblem(logistic_model, u0_logistic, tspan_logistic, p_logistic)
sol_logistic = solve(prob_logistic)

# График для задания 2
plot(sol_logistic, linewidth=2, title="Логистическая модель роста",
     xaxis="Время", yaxis="Численность популяции", label="Численность",
     color=:blue)
hline!([k], linewidth=2, linestyle=:dash, label="Ёмкость среды K=$k", color=:red)
```

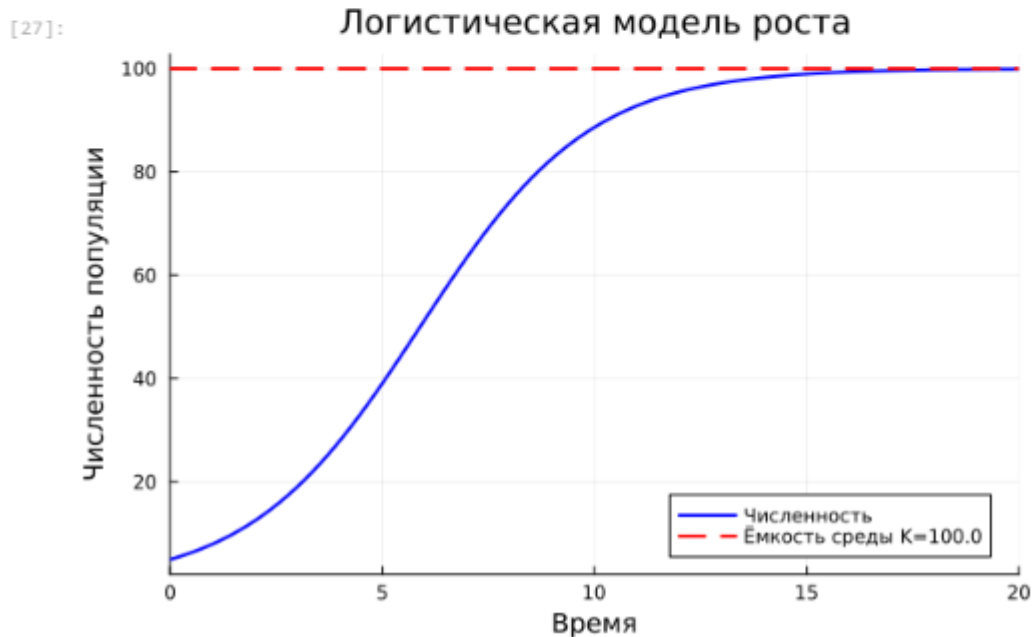


Рис. 8: Решение задания №2

Выполнение задания №3 (рис. 9):

```
[39]: # ЗАДАНИЕ 3: Модель эпидемии SIR

function sir_model!(du, u, p, t)
    s, i, r = u
    β, v = p

    du[1] = -β * s * i      # ds/dt
    du[2] = β * s * i - v * i # di/dt
    du[3] = v * i           # dr/dt
end

# Параметры
β = 0.3 # коэффициент заражения
v = 0.1 # коэффициент выздоровления
N = 1000 # общая численность популяции

# Начальные условия: 1 зараженный, остальные восприимчивые
#u0_sir = [N-1.0, 1.0, 0.0]
u0_sir = [0.99, 0.01, 0.0]
tspan_sir = (0.0, 100.0)
p_sir = (β, v)

prob_sir = ODEProblem(sir_model, u0_sir, tspan_sir, p_sir)
sol_sir = solve(prob_sir)

# График для задания 3 (исправленная версия)
plot(sol_sir, linewidth=2, title="Модель эпидемии SIR",
     xaxis="Время", yaxis="Численность",
     label=["Восприимчивые" "Зараженные" "Выздоровшие"],
     color=[:blue :red :green])
```

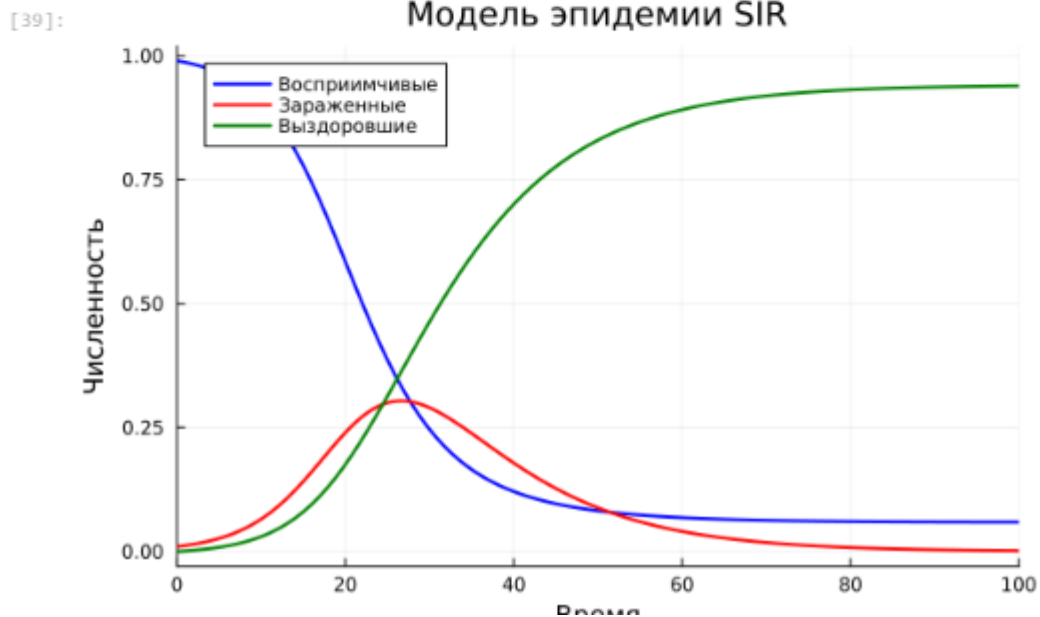


Рис. 9: Решение задания №3

Выполнение задания №4 (рис. 10):

```
[68]: # ЗАДАНИЕ 4: Модель SEIR

function seir_model(du, u, p, t)
    s, e, i, r = u
    β, δ, γ, N = p

    du[1] = -β/N * s * i      # ds/dt -
    du[2] = β/N * s * i - δ * e # de/dt -
    du[3] = δ * e - γ * i      # di/dt -
    du[4] = γ * i              # dr/dt -
end

# Параметры
β = 0.3 # коэффициент заражения
δ = 0.1 # скорость перехода из экспонированных в зараженные
γ = 0.1 # скорость выздоровления
N = 1.0 # общая численность

# Начальные условия
u0_seir = [0.99, 0.0, 0.01, 0.0]
tspan_seir = (0.0, 100.0)
p_seir = (β, δ, γ, N)

prob_seir = ODEProblem(seir_model, u0_seir, tspan_seir, p_seir)
sol_seir = solve(prob_seir, Tsit5())

# График для задания 4 - сравнение SIR и SEIR
plot(sol_seir, label=["Восприимчивые", "Экспонированные", "Инфицированные", "Выздоровевшие"],
     title="Модель эпидемии SEIR", xlabel="Время", ylabel="Численность", linewidth=2)
```

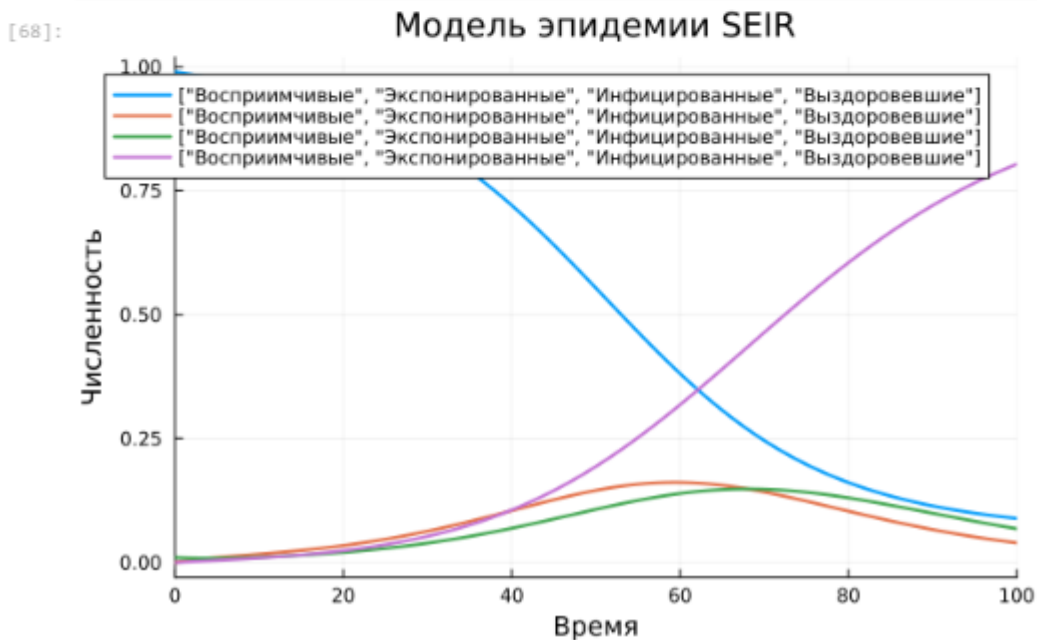


Рис. 10: Решение задания №4

Выполнение задания №5 (рис. 11):

```
[72]: # ЗАДАНИЕ 5: Дискретная модель Лотки-Вольтерры

function lotka_volterra(X, p)
    a, c, d = p
    x1, x2 = X

    x1_next = a * x1 * (1 - x1) - x1 * x2
    x2_next = -c * x2 + d * x1 * x2

    return [x1_next, x2_next]
end

# Параметры
a = 2.0
c = 1.0
d = 5.0

# Начальные условия
X0 = [0.1, 0.1]

# Численное решение на 100 шагов
T = 100
dt = 0.1
times = 0:dt:T

X = zeros(length(times), 2)
X[1, :] = X0

for i in 2:length(times)
    X[i, :] = X[i-1, :] + dt * lotka_volterra(X[i-1, :], (a, c, d))
end

# График для задания 5 - фазовый портрет
plot(X[:,1], X[:,2], label="Фазовый портрет", xlabel="X1", ylabel="X2")
```

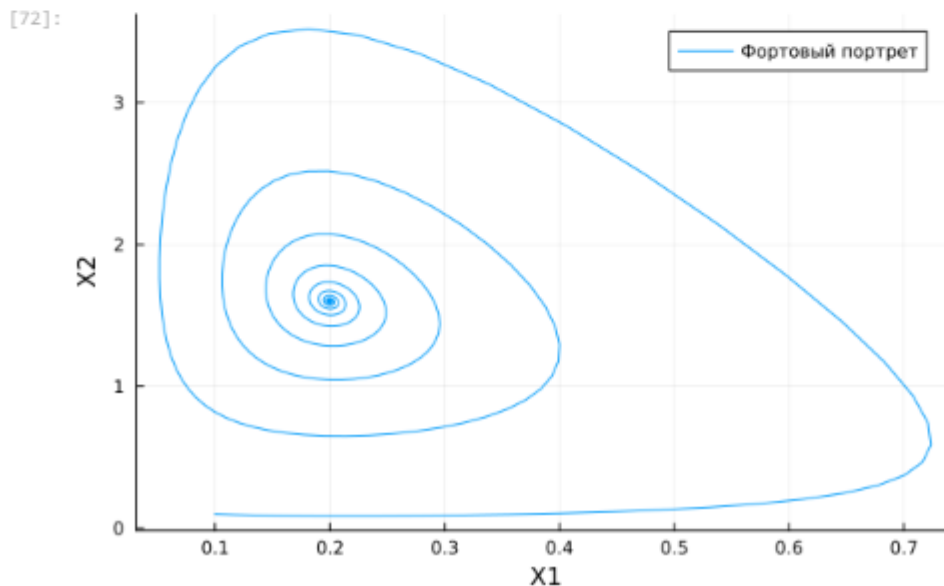


Рис. 11: Решение задания №5

Выполнение задания №6 (рис. 12):

```
[80]: # ЗАДАНИЕ 6: Модель конкурентных отношений

function competition_model(du, u, p, t)
    α, β = p
    x, y = u

    du[1] = α * x - β * x * y
    du[2] = α * y - β * x * y
end

# Параметры
α = 0.1 # коэффициент роста
β_ = 0.01 # коэффициент конкуренции
p = (α, β)
u0 = [10.0, 5.0] # начальные численности
tspan = (0.0, 100.0)

prob = ODEProblem(competition_model, u0, tspan, p)
sol = solve(prob, Tsit5())

# График для задания 6
plot(sol, label=["Популяция X", "Популяция Y"], title="Модель конкуренции", xlabel="Время", ylabel="Популяция", linewidth=2)
```

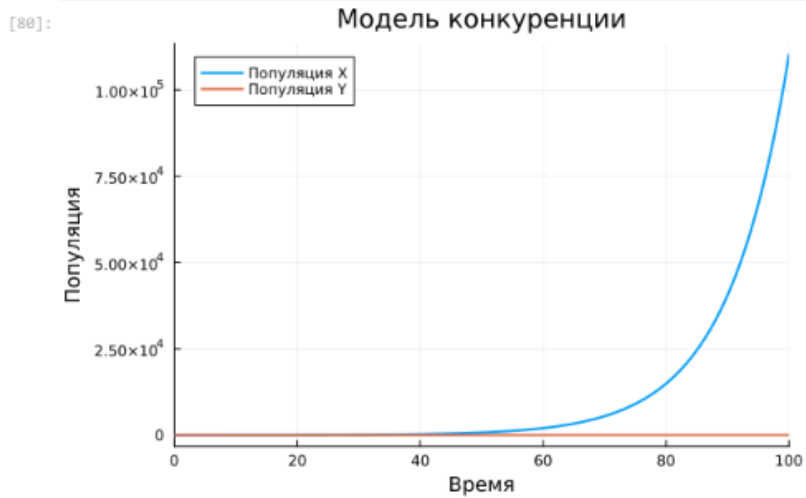


Рис. 12: Решение задания №6

Выполнение задания №7 (рис. 13):

```
[86]: # ЗАДАНИЕ 7: Консервативный гармонический осциллятор

function harmonic_oscillator(du, u, p, t)
    w0 = p[1]

    du[1] = u[2]
    du[2] = -w0^2 * u[1]
end

# Параметры
x0 = 1.0 # циклическая частота (период = 1)
y0 = 0.0 # начальное смещение 1, начальная скорость 0
u0 = [x0, y0]

w0 = 1.0
p = [w0]

tspan = (0.0, 50.0)

prob = ODEProblem(harmonic_oscillator, u0, tspan, p)
sol = solve(prob, Tsit5())

# График для задания 7
plot(sol, label=["Положение X" "Скорость X"], title="Гармонический Осциллятор", xlabel="Время", ylabel="Значение", linewidth=2)
plot(sol[1, :], sol[2, :], label="Фазовый портрет", xlabel="Положение X", ylabel="Скорость X", linewidth=2)
```

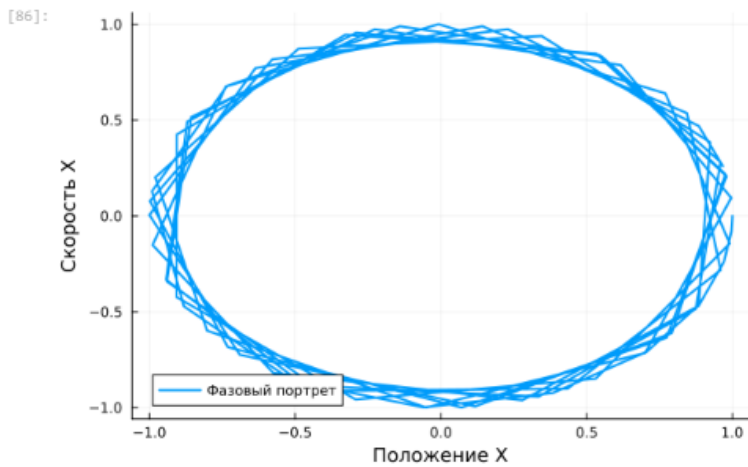


Рис. 13: Решение задания №7

Выполнение задания №8 (рис. 14):

```
[100]: # ЗАДАНИЕ 8: Свободные колебания гармонического осциллятора

function damped_oscillator(du, u, p, t)
    Y, w0 = p
    du[1] = u[2]
    du[2] = -2 * Y * u[2] - w0^2 * u[1]
end

# Параметры
u0 = [1.0, 0.0]
Y = 0.1
w0 = 1.0
p = [Y, w0]

tspan = (0.0, 50.0)

prob = ODEProblem(damped_oscillator, u0, tspan, p)
sol = solve(prob, Tsit5())

# График для задания 8
plot(sol, label=["Положение X" "Скорость X'"], title="Свободное колебание с потерями энергии", xlabel="Время", ylabel="Значение", linewidth=2)
```

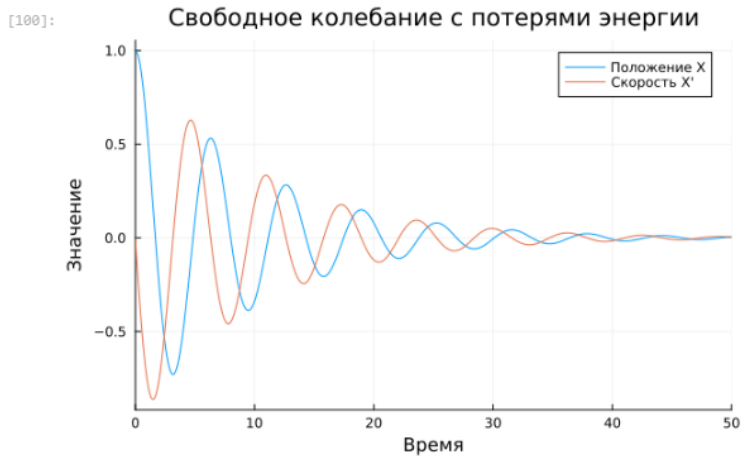


Рис. 14: Решение задания №8

3 Вывод

В ходе выполнения лабораторной работы были освоены специализированные пакеты для решения задач в непрерывном и дискретном времени.

4 Список литературы. Библиография

[1] Julia Documentation: <https://docs.julialang.org/en/v1/>